

BINV2150-A – Javascript : avancé

Séance 4 – API REST

Jérome Plumat & Sébastien Strebelle

Haut École Léonard de Vinci

2025-2026



Haute École Léonard de Vinci

API



- API = Application Programming Interface
- Permet à des applications de communiquer entre elles.
- Souvent via HTTP (REST, GraphQL, SOAP, gRPC, etc.)
- Utilisé pour les applications web, mobiles, etc.

REST



- REST = REpresentational State Transfer
- Style d'architecture pour les API web.

- Utilise les méthodes HTTP (GET, POST, PUT, DELETE, etc.)
- Ressources identifiées par des URLs.
- Stateless : chaque requête est indépendante.

Structure d'un message HTTP

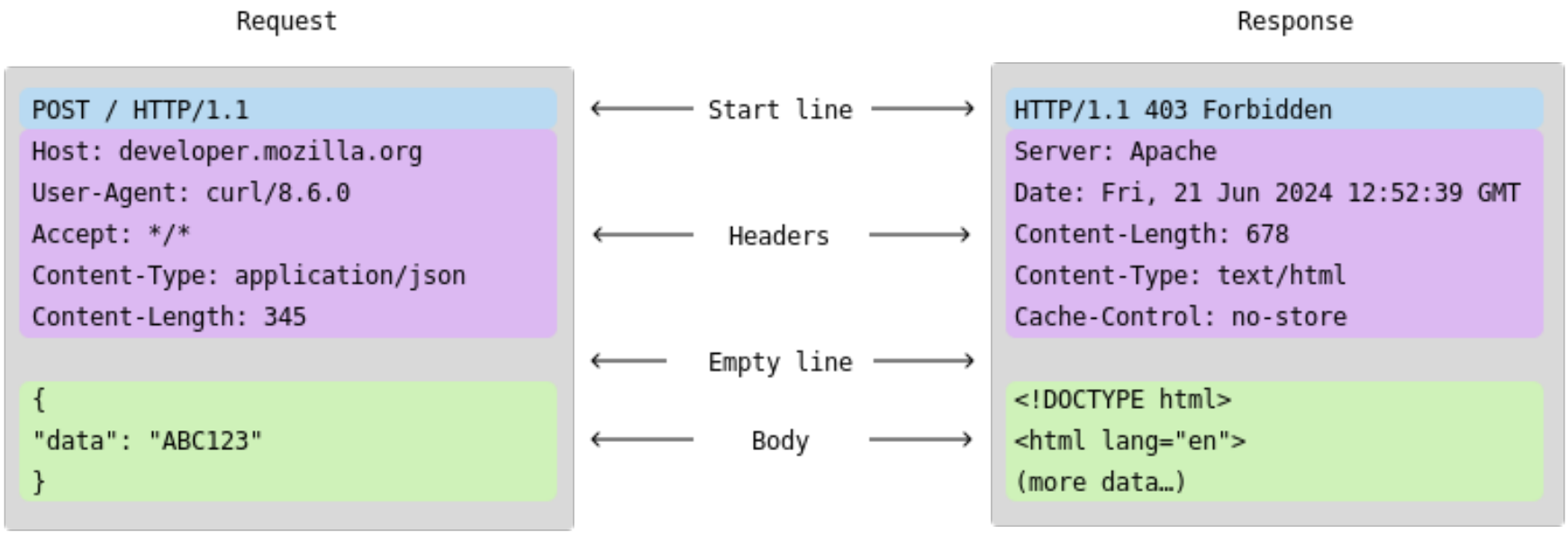


Figure – Structure d'un message HTTP, source : developer.mozilla.org

Ressources



- Ressource = entité manipulée par l'API.
- Identifiée par une URL : correspond au nom de l'entité, **au pluriel**.
- Exemples :
  - /users : Utilisateurs
  - /products : Produits
  - /orders : Commandes
- Peut aussi inclure des sous-ressources :
  - /users/1/orders : Commandes de l'utilisateur avec ID 1.
- En express : `app.get('/users', ...)`, `req.path`

Méthodes HTTP



- Méthode = ce qu'on veut faire sur la ressource.

GET	Récupérer une ressource ou une liste de ressources.
POST	Créer une nouvelle ressource.
PUT	Mettre à jour une ressource existante (remplacement complet).
PATCH	Mettre à jour partiellement une ressource.
DELETE	Supprimer une ressource.

- En express : `app.get(...)` , `req.method`

Méthode et chemin



- Combinaison méthode + chemin = action sur une ressource.
- Exemples :
  - GET /users : Récupérer tous les utilisateurs.
  - GET /users/1 : Récupérer l'utilisateur avec l'ID 1.
  - POST /users : Créer un nouvel utilisateur.
  - PUT /users/1 : Mettre à jour l'utilisateur avec l'ID 1.
  - DELETE /users/1 : Supprimer l'utilisateur avec l'ID 1.

Headers



- Métadonnées sur la requête ou la réponse.
- Exemples de headers courants :
  - Content-Type : Type de contenu (ex : application/json).
  - Authorization : Informations d'authentification (ex : token).
  - Accept : Types de contenu acceptés par le client.
- En express : `req.headers['content-type']`, `req.headers.authorization`

Paramètres de chemin

Path parameters



- Paramètres dans le chemin de la requête pour identifier une ressource spécifique.
- Exemples :
  - /users/1 : Utilisateur avec ID 1.
  - /products/42 : Produit avec ID 42.
- En express : `app.get('/users/:id', ...)` , `req.params.id`

Paramètres de requête

Query parameters



- Paramètres ajoutés à l'URL après un ? pour filtrer ou modifier la requête.
- Exemples :
  - /users ?age=30 : Utilisateurs âgés de 30 ans.
  - /products ?category=electronics : Produits dans la catégorie "electronics".
  - /orders ?status=pending&sort=asc : Commandes avec le statut "pending", triées par ordre croissant.
- En express : `req.query.age`, `req.query.category`, etc.

Corps de la requête

Request body



- Données envoyées au serveur dans le corps de la requête en JSON.
- Utilisé principalement avec POST, PUT, PATCH.
- Exemples :
  - POST /users avec `{ "name": "John", "age": 30 }` : Créer un nouvel utilisateur.
  - PUT /users/1 avec `{ "name": "Jane", "age": 25 }` : Mettre à jour l'utilisateur avec ID 1.
- En express : nécessite un middleware comme `express.json()` pour parser le JSON, puis accès via `req.body`.

Statut de la réponse



- Code de statut HTTP pour indiquer le résultat de la requête.
- En express : `res.status(200).json(data)`, `res.sendStatus(404)`, etc.
- Codes de succès :
  - 200 OK : Requête réussie, réponse renvoyée au format JSON.
  - 201 Created : Ressource créée avec succès (POST).
  - 204 No Content : Requête réussie, mais pas de contenu à renvoyer.

Codes d'erreur



400 Bad Request : Requête invalide. Souvent dû à des données manquantes ou incorrectes.

401 Unauthorized : Non autorisé. L'utilisateur n'est pas correctement authentifié.

403 Forbidden : Interdit. L'utilisateur est authentifié mais n'a pas les droits.

404 Not Found : Ressource non trouvée. L'URL est incorrecte ou la ressource n'existe pas.

409 Conflict : Conflit. Souvent pour indiqué que la ressource existe déjà (POST).

500 Internal Server Error : Erreur serveur. Le traitement de la requête a mené à une erreur non traitée.

Résumé



- API REST = interface pour manipuler des ressources via HTTP.
- Ressources identifiées par des URLs.
- Méthodes HTTP pour définir les actions (GET, POST, PUT, DELETE, etc.)
- Paramètres de chemin et de requête pour filtrer et identifier les ressources.
- Corps de la requête pour envoyer des données (JSON).
- Codes de statut HTTP pour indiquer le résultat.